Why Risk it, When You Can {rix} it: A Tutorial for Computational Reproducibility Focused on Simulation Studies

Felipe Fontana Vieira[1], Jason Geller[2], and Bruno Rodrigues[3]

[1]Department of Data Analysis, Ghent University

[2]Department of Psychology and Neuroscience, Boston College

[3]Statistics and Data Strategy Departments, Ministry of Research and Higher Education, Luxembourg

Author Note

Felipe Fontana Vieira https://orcid.org/0009-0006-0949-6569

Jason Geller https://orcid.org/0000-0002-7459-4505

Bruno Rodrigues https://orcid.org/0000-0002-3211-3689

Data and materials from this study can be accessed at https://doi.org/10.5281/zenodo.21098114. Bruno Rodrigues is one of the authors and maintainers of the {rix} package presented in this tutorial. The authors have no other conflicts of interest to declare. We would like to thank Julia Rohrer, Ole Schacht and Dries Debeer for their helpful feedback on earlier versions of this manuscript, and Matteo Quartagno for feedback on seeds and reproducibility. During the first round of revision, Claude Sonnet 4.6 (Anthropic) was used to improve the grammar and coherence solely of the Introduction and Discussion. Author roles were classified using the Contributor Role Taxonomy (CRediT; https://credit.niso.org/) as follows: Felipe Fontana Vieira: Conceptualization, Methodology, Software, Data curation, Formal analysis, Visualization, Writing - original draft, Writing - review & editing; Jason Geller: Conceptualization, Supervision, Validation, Writing - review & editing; Bruno Rodrigues: Conceptualization, Software, Writing - review & editing

Correspondence concerning this article should be addressed to Felipe Fontana Vieira, Email: felipe.fontanavieira@ugent.be

Abstract

Computational reproducibility remains limited in psychological research, despite widespread norms for sharing data and analysis code. One reason is that reproducibility exists on a continuum, ranging from partial transparency—such as providing scripts or software version numbers—to fully executable research compendia that regenerate all results from raw code. In this article, we introduce Nix and the {rix} R package as a practical framework for achieving full computational reproducibility in simulation-based research. We provide a step-by-step tutorial demonstrating how {rix} can be used to define, build, and share isolated, project-specific software environments that precisely capture R versions, package dependencies, system libraries, and integrated development environments. We further illustrate this workflow by reproducing a complete manuscript using Quarto and the {apaquarto} extension, showing how analyses, figures, and text can be regenerated in a single, executable pipeline. Together, these tools lower the technical barrier to robust, end-to-end reproducibility and offer a scalable solution for simulation studies and methodological research in psychology and related fields.

Keywords: reproducibility, Nix, simulation studies, R, computational methods

Why Risk it, When You Can {rix} it: A Tutorial for Computational Reproducibility Focused on Simulation Studies

Psychological science is in the midst of a credibility revolution (Vazire, 2018). However, to date, much of the attention has focused on empirical studies. Monte Carlo simulation studies (hereafter referred to as simulation studies) have received far less (Siepe et al., 2024), even though they are a primary tool for evaluating the statistical methods (Morris et al., 2019) and may suffer from selective reporting (Bouma et al., 2026; Pawel et al., 2024, 2025). One underexamined aspect for simulations is reproducibility (i.e., exactly reproducing a study's results from the provided data, code, materials, and software) (Hardwicke et al., 2020). In a simulation study, this involves more than reanalyzing a fixed dataset, because the code also generates the data. Reproducing such a study therefore involves the data generation process, the analysis, and the summary of results. Montoya and Anderson (2026) conceptually address each of these stages.

Reproducibility in this sense is not all-or-nothing but occupies a continuum (Peng, 2011), on which a study is positioned according to the extent of the materials that are publicly available. This ranges from publication alone at the low end to code and data that are linked and directly executable at the high end. Reaching that executable end is often treated as *full* reproducibility (Peikert et al., 2021; Peikert & Brandmaier, 2021), and open science initiatives have pushed work toward it (Kidwell et al., 2016; Levenstein & Lyle, 2018; Nosek et al., 2015). Yet code and data are never fully sufficient (Brodeur et al., 2026; Epskamp, 2019; Wiebels & Moreau, 2021; Ziemann et al., 2023). Even error-free code depends on a hierarchy of software components, including the programming language version, the packages used in the analysis, and the system-level libraries those packages rely on. We define these and related terms in Table 1. When they differ from the originals, code may fail, behave inconsistently across machines, or yield conflicting numerical results (Baker et al., 2024; Glatard et al., 2015; Hodges et al., 2023; Nosek et al., 2022). Such problems are particularly acute for simulation studies, which rely on interconnected code, package versions, and the broader software environment to generate their data and results (Luijken et al., 2024; Siepe et al., 2024).

To make this concrete, we use the *computational environment* to refer to the complete software context required for a study to run successfully. This entails the programming language version, package versions, system-level libraries, and operating system (Figure 1) (Rodrigues, 2023; Rodrigues & Baumann, 2026). We define *computational environment reproducibility* as the ability to reconstruct these dependencies on top of whatever operating system a machine runs, at any future time, such that executing the same code yields the same numerical results[1]. Current practice falls short. Indeed, Siepe et al. (2024) report that, among the 100 simulation studies they identified in psychology journals in 2021 and 2022, nearly two-thirds provide no code, and those that do rarely document the computational environment. This gap is consequential: because simulation studies inform methodological recommendations, insufficient reproducibility may undermine confidence in them (Luijken et al., 2024; White et al., 2024).

These challenges may persist for several reasons. Arguably, one is that even researchers aware of these demands face a fragmented landscape of solutions, each addressing only part of the problem. Package-level managers such as {renv} (Ushey, 2024) and {groundhog} (Simonsohn, 2020) stabilize R package versions but do not manage the R interpreter itself or the system-level libraries those packages depend on. Analytical pipeline tools such as {targets} (Landau, 2021) and Make (Feldman, 1979) support reproducibility in a different sense: they specify and automate the structure of an analysis by formalizing the order in which steps should run and by tracking dependencies among intermediate results. These tools clarify *how* an analysis proceeds, but they assume that the software stack required to run each step is already stable. Containerization tools such as Docker, including R-focused implementations like Rocker (Boettiger, 2015; Boettiger & Eddelbuettel, 2017) that provide preconfigured Docker images tailored to common R workflows, offer a more comprehensive approach by bundling nearly the entire environment into a single executable image. Yet their use requires familiarity with Linux system administration, and even containerization may suffer from temporal drift when Dockerfiles rely on mutable upstream

[1] We note our characterization of computational environment reproducibility is not the most rigorous. For stricter definitions, we refer the interested reader to Malka et al. (2026) (p. 3) and Lamb and Zacchiroli (2022).

repositories (Malka et al., 2026). For a detailed comparison of these tools and their limitations, see Rodrigues and Baumann (2026). Researchers thus face a trade-off between accessible but incomplete solutions and powerful but technically demanding ones.

In this article, we focus specifically on computational environment reproducibility as the foundation upon which other reproducibility practices depend. For that, we introduce Nix (Dolstra et al., 2004), a software ecosystem designed to make software installation deterministic, and {rix} (Rodrigues & Baumann, 2025), an R interface that allows researchers to use Nix without needing deep knowledge of its underlying language or infrastructure. Our main objective with the tutorial is not to advocate for specific analytical pipeline tool or to prescribe a particular workflow structure[2]. Instead, we aim to show what Nix and {rix} are and how they can establish a stable environment within which simulation studies—whether organized in `.R` files that `source()` others, through formal pipeline tools (e.g., {targets}) or embedded as code chunks in `.Rmd` or `.qmd` files—can be executed reliably. We illustrate these ideas through a reproducible simulation study conducted in R, culminating in this automated APA-formatted manuscript generated with `apaquarto` (Schneider, 2024). Although the example centers on R, the principles underlying environment reproducibility with {rix} apply equally to other languages, including Python and Julia, and to multiple development environments such as RStudio, VS Code, or Positron.

The remainder of this article is structured as follows. We first illustrate a simulation study and the reproducibility risks associated with it. We then introduce Nix and {rix}, along with their core principles. This is followed by a step-by-step tutorial that (re)builds the simulation study as a reproducible project with {rix}, from installing Nix to (re)generating this manuscript. Last, we turn to additional considerations for reproducibility of simulation studies and close with a discussion.

[2] The analytical pipeline is one part of the broader reproducible workflow. We leave it aside for two reasons. First, many tools address it, each with its own conventions, and doing them justice would require a more tailored treatment. Second, although a formal analytical pipeline may be necessary for full reproducibility, it rests on a stable computational environment. Arguably, a study whose scripts are well documented, whose file dependencies are explained, and whose environment is stable therefore already sits high on the reproducibility spectrum, and that foundation is what we focus on here. For readers interested in the broader reproducible workflow, we suggest Peikert et al. (2021), van Lissa et al. (2021), and Rodrigues (2023).

Setting the Stage: A Simulation Study and Its Reproducibility Risks

Imagine you have just been awarded a grant to conduct a large-scale simulation study, where you want to evaluate the performance of a statistical estimator under varying data-generating conditions (see Appendix A for full technical details). This tutorial is organized around this example to ground our discussion in a typical methods section, but readers can follow the tutorial without engaging deeply with the simulation itself. In practice, simulation studies are typically organized into multiple component files. To mimic this practice, our simulation is organized into five sequential scripts: data generation (`01_data_generation.R`), model specification (`02_models.R`), simulation execution (`03_run_simulation.R`), performance metric calculation (`04_performance_metrics.R`), and results visualization (`05_plots.R`) [3].

Now suppose a researcher attempts to reproduce the simulation run and results reported in this article. What might prevent them from obtaining identical outcomes, assuming error-free code? The natural first concern is the version of packages. Installing R packages at a later time may lead to errors if functions have been renamed or deprecated (e.g., `dplyr::cur_data()` changed to `pick()`, or, although usually discouraged, internal functions, such as `lavaan:::lav_utils_get_ancestors`, renamed to `lavaan:::lav_graph_get_ancestors`), or to subtle differences in the results due to changes in default settings (e.g., `stringsAsFactors` defaulting to FALSE as of R 4.0). Beyond package versioning, many packages rely on system-level software that R does not provide. Our simulation illustrates this directly: the {rvinecopulib} package is built on a compiled C++ backend (Nagler & Vatter, 2025).

The R language version introduces another layer of dependency. Code written for a recent version of R may rely on syntax or functionality that is unavailable in earlier versions (e.g., the native pipe `|>` introduced in R 4.1). More subtly, changes to R's random number generation across versions mean that identical code executed with the same seed can nevertheless produce different

[3] For simplicity, we included all code as executable chunks in a single `.qmd` file. This entails that the simulation runs from start to finish (i.e., generating data, producing results and figures). This is impractical for many real-world simulation studies, which are often too computationally intensive. One possible solution, which we illustrate, is to run the simulation outside the `.qmd` file and load the results for the analyses in the same file.

random sequences (Ottoboni & Stark, 2018). This is especially consequential for simulation studies, because the random draws are not incidental to the analysis but constitute the data itself. Therefore, the random number generator is part of the computational environment. Moreover, generating data and reproducing every statistical estimate *exactly* (down to the certain digits) can depend on lower-level components still, such as the system-level libraries R uses for matrix computations (e.g., BLAS/LAPACK). The potential solution we propose pins these libraries, and increases the chances of reaching such a level of reproducibility, consistent with empirical findings in other contexts (Malka et al., 2026). However, it must be noted that reproducing precisely every number is hard to guarantee in every case, especially across different operating systems and processors (e.g., Stan Development Team, 2025). Although such differences are usually negligible, appearing only in the last digits, the finite precision of floating-point arithmetic (Ahrens et al., 2020; Goldberg, 1991) and the algorithm a specific R package uses to generate data can occasionally amplify them into larger discrepancies. With respect to all of this, we do not claim that this exact level of identity must be met. How much reproducibility is required is ultimately for the scientific community to decide (e.g., through journals and disciplinary norms), and reproducing the broader pattern of results is arguably enough (see also Montoya and Anderson (2026), for a stage-by-stage approach to assessing reproducibility in simulation studies)[4].

Finally, analyses embedded in a literate programming workflow add further dependencies. Rendering R Markdown (`.Rmd`) or Quarto (`.qmd`) documents, for example, requires a document conversion tool (such as Pandoc) and a typesetting system (such as LaTeX or Typst), each with its own version and installation requirements.

[4] The following illustration originated from Danielle Navarro. With the same seed and covariance matrix, `MASS::mvrnorm()` can return different values on macOS than on Linux or Windows, whereas `mvtnorm::rmvnorm()` stays consistent. The difference traces to a sign flip in an eigenvector computed through the system's BLAS/LAPACK libraries, to which `mvtnorm` is invariant but `MASS` is not. The draws still come from the same distribution, so conclusions would generally be unchanged in simulation with multiple repetitions, but from a *pure* reproducibility standpoint this is concerning. Pinning the numerical library with Nix (here, OpenBLAS) removes the discrepancy for `MASS::mvrnorm()`. For the full demonstration, see the repository.

Nix and {rix}: A Comprehensive Solution

A potential solution is Nix (Dolstra et al., 2004), a software ecosystem built so that installing software gives the same result every time. Nix is a functional programming language, a build tool, and a package manager wrapped in one. This article focuses on the latter. Most package managers (think of Apple's or Android's app stores) are imperative: they modify a system's state as they install or update software. Nix instead builds each environment from an explicit description and never alters it afterward, so the same description produces the same self-contained environment on other machines. Researchers state which version of R and which packages they need, and Nix works out the underlying system-level libraries automatically.

Core Principles

Rather than installing software into shared system folders (e.g., `/usr/lib`), Nix gives every package its own folder under `/nix/store`. The folder's name includes a short fingerprint (a *cryptographic hash*) computed from everything that went into the package, its source code, dependencies, and build instructions, so any change to those inputs yields a different folder. Because each package is identified by its exact inputs this way, several versions of the same software can sit side by side without conflict: a researcher can, for example, keep R 4.1.0 and R 4.3.3 on the same machine, or use different package versions across analyses (Rodrigues & Baumann, 2025).

The Nix ecosystem is built around `nixpkgs`, a version-controlled repository comprising more than 120,000 packages, including nearly all of CRAN and Bioconductor. By pinning a specific commit or date, researchers freeze the entire software stack (i.e., R itself, R packages, and all system-level libraries). This eliminates the system-dependency problems that tools like {renv} cannot address (Rodrigues & Baumann, 2025). Empirical studies of historical nixpkgs snapshots report that old environments can still be rebuilt years later (Malka et al., 2024; Rodrigues & Baumann, 2026). Moreover, because prebuilt binaries can be downloaded from a shared binary cache instead of compiled from scratch, these environments are often ready in seconds, which keeps Nix practical for everyday interactive work (Rodrigues & Baumann, 2025).

The {rix} Package: R Interface to Nix

Because the Nix package manager is declarative, installing software requires writing expressions in its language. Since this language is unfamiliar to most researchers, we recommend and focus on using {rix} to lower this barrier. The {rix} package provides an R-native interface: a single call to `rix()` generates complete Nix configurations from standard R syntax, specifying R versions, CRAN packages, system-level software, and even Python or Julia components when required. Users never need to read or write Nix code directly, as {rix} translates automatically. For more on {rix}, including its integration with rstats-on-nix (a curated fork of the Nix package collection whose dated snapshots let users request the exact R and package versions available on a chosen day), see Rodrigues and Baumann (2025) and Rodrigues and Baumann (2026).

A Practical Example: Setting up a Reproducible Simulation Study with {rix}

We take the perspective of a researcher (e.g., on macOS) beginning a simulation study who wants it reproducible from the outset. Thus, we set up the computational environment with {rix} first, then create and run the scripts, and manuscript inside it. Last, we show how another researcher could reproduce it.

Step 1: Installing Nix and {rix}

Both Nix and the {rix} R package need to be installed. The quick start below covers the common case, but the complete, up-to-date instructions are maintained in the {rix} documentation: Linux and Windows (https://docs.ropensci.org/rix/articles/setting-up-linux-windows.html) and macOS (https://docs.ropensci.org/rix/articles/setting-up-macos.html).

The installation is the same across operating systems, with one prerequisite on Windows: Nix runs inside the Windows Subsystem for Linux 2 (WSL2), which must be installed and configured first (see the {rix} documentation above). On all systems, install Nix by running the Determinate Systems installer in a terminal[5]. To open it, on macOS, go to Applications →

[5] A terminal is a text-based way to give your computer instructions. Readers new to the command line may find the following link a useful primer: https://swcarpentry.github.io/shell-novice/. This was also suggested in Wiebels and

Utilities → Terminal. For Windows, launch your WSL2 (Ubuntu) shell from the Start menu. For Linux, open your distribution's terminal application. Then run (Listing 1):

Listing 1 Installing Nix

```
curl --proto '=https' --tlsv1.2 -sSf \
  -L https://install.determinate.systems/nix | \
  sh -s -- install
```

Once Nix is installed[6], there are two ways to access {rix}, depending on whether R is already installed on your system. In this tutorial, we proceed as if R was already installed[7], as that most likely will be the case for most readers (Listing 2):

Listing 2 Installing {rix} from CRAN or developmental version

```
# CRAN version
install.packages("rix")
# Development version
install.packages(
  "rix",
  repos = c(
    "https://ropensci.r-universe.dev"
  )
)
```

Step 2: Specifying the Computational Environment With `rix()`

We first begin by creating a folder for this project (here named `Why-risk-it-when-you-can-rix-it`) that contains only two empty subfolders, `Simulation` and `Manuscript`. We have not written any simulation scripts or the manuscript yet. The only file

Moreau (2021)

[6] It is worth noting that {rix} can generate Nix expressions even without Nix installed on your system: you can write a `default.nix` file without Nix, but you cannot build or enter the resulting environment unless Nix is installed (Rodrigues & Baumann, 2025).

[7] We, however, recommend uninstalling your local R and letting Nix manage R, R packages, and other tools entirely. This approach avoids potential conflicts between system-installed and Nix-managed software, an issue we will illustrate later in this tutorial. See the {rix} documentation for more details.

we write by hand for now is `gen-env.R`, which uses the `rix()` function from the {rix} package to produce a `default.nix` file, a declarative specification of all software dependencies the project needs.

We establish the full environment the simulation needs up front: the R version, an IDE, and the R packages the scripts will use. You can always add more later, but doing so means editing `gen-env.R` and following the steps we are about to illustrate. The simulation specification is shown in Listing 3 (the complete version, which also covers the manuscript, is in the project repository, https://doi.org/10.5281/zenodo.21098114, as `gen-env.R`):

Listing 3 Simulation environment: R, RStudio, and the packages the scripts need

```
library(rix)

# choose the path to your project
# here, the "Why-risk-it-when-you-can-rix-it" folder (root)
project_directory <- "."

rix(
  date = "2026-01-14", # recommendation: pin a date
  r_pkgs = c(
    # all R packages the simulation scripts need
    "rix",
    "marginaleffects",
    "simhelpers",
    "rvinecopulib",
    "doParallel",
    "doRNG",
    "ggplot2",
    "cowplot",
    "dplyr",
    "svglite"
  ),
  ide = "rstudio", # editor to include
  project_path = project_directory,
  overwrite = TRUE
)
```

Environment Specification

The `rix()` function[8] constructs the specification through a series of parameters that collectively describe the computational environment. Each parameter serves a distinct purpose in defining the environment's characteristics, and we will clarify those throughout this tutorial section.

Specifying the R version. Researchers must first determine which version of R to use. This can be accomplished in two ways: The `r_ver` argument accepts an exact version string (e.g., "4.3.3") or special designations such as "latest-upstream" for the most recent stable release. Alternatively, the `date` argument specifies a particular date (e.g., "2024-11-15"), which ensures that R and all packages correspond to the versions available on that date. The date-based approach is generally preferable for reproducibility, as it captures a complete snapshot of the R ecosystem at a single point in time. For this tutorial, as shown on top, we use the `date` parameter to ensure temporal consistency across all software components (Rodrigues & Baumann, 2025).

Listing 4 Example for the date argument

```
date = "2026-01-14"
```

Declaring R package dependencies. The `r_pkgs` argument accepts a character vector listing all required R packages by their CRAN names. These packages will be installed from the version repository corresponding to the specified date or R version. It is important to list all packages that the analysis will load directly; dependencies of these packages are automatically resolved by Nix.

For packages requiring specific versions not corresponding to the chosen date, researchers can specify exact versions using the syntax `"packagename@version"` (e.g., `"ggplot2@2.2.1"`). For packages available only on GitHub or other Git repositories, the `git_pkgs` argument accepts a list structure containing repository URLs and specific commit hashes. For example:

[8] For an overarching information on the function `rix()` and more details, we suggest the following {rix} documentation: https://docs.ropensci.org/rix/articles/project-environments.html

Listing 5 Example for the r_pkgs argument

```
r_pkgs = c(
  "rix", "marginaleffects", "simhelpers", "rvinecopulib",
  "doParallel", "doRNG", "ggplot2", "cowplot", "dplyr", "svglite"
)
```

Listing 6 Example for the git_pkgs argument

```
git_pkgs = list(
  package_name = "marginaleffects",
  repo_url = "https://github.com/vincentarelbundock/marginaleffects",
  commit = "304bff91dc31ae28b227a8485bfa4f7bdc86d625"
)
```

This ensures that exact development versions are obtained (Rodrigues & Baumann, 2025). For our simulation study, all packages were used with their CRAN versions. Appendix B gives the reasons for adding each package in `r_pkgs`.

Configuring the development environment. The `ide` parameter controls whether an integrated development environment (IDE) is included in the Nix environment, allowing users to interactively develop and run code within their editor of choice. When `ide` is specified, the project can be opened directly in the corresponding IDE, with all dependencies provided by the Nix environment. Most readers likely already have an IDE installed locally. Here we instead show how to open RStudio from the rix specification, which does not require RStudio to be installed on your system. For example, setting `ide = "rstudio"` installs a project-specific version of RStudio inside the Nix environment. This is required for RStudio because, unlike most other editors, it cannot attach to an external Nix shell unless it is itself installed via Nix. On macOS, RStudio is only available through Nix for R versions 4.4.3 or later (or environments dated 2025-02-28 or later). For earlier R versions, alternative editors must be used. Other supported IDEs include Positron (`ide = "positron"`), Visual Studio Code (`ide = "code"`), and command-line interfaces such as Radian (`ide = "radian"`). These tools may either be installed directly within the Nix environment using the `ide` parameter, or users may rely on an existing

system installation by setting `ide = "none"` (or `ide = "other"`) and configuring `direnv` to automatically load the Nix environment when the project directory is opened[9]. All IDEs installed via Nix are project-specific and do not interfere with system-wide installations.

Listing 7 Example for the ide argument

```
ide = "rstudio"
```

Setting file output parameters. The `project_path` parameter indicates where the `default.nix` file should be written (“.” denotes the current directory), while `overwrite` controls whether an existing file should be replaced. Adding to this, setting `print = TRUE`, which is another argument, displays the generated specification in the console for immediate verification (Rodrigues & Baumann, 2025).

Listing 8 Example for the project_path and overwrite arguments

```
project_path = ".",
overwrite = TRUE
```

Step 3: Generating the Environment Specification

After defining the computational environment, the `rix()` function must be executed to generate the `default.nix` file. This can be done interactively by running `rix()` (Listing 3). The file is written to the folder given in `project_path`, so you can confirm it succeeded by checking that this folder now contains `default.nix`.

Step 4: Building and Using the Reproducible Environment

Once Step 3 is complete, we build the reproducible environment by navigating to the project directory in the terminal. In our case, this is the `Why-risk-it-when-you-can-rix-it` folder. From there, run the following command (Listing 9):

[9] `direnv` is a lightweight utility that integrates with the user’s shell and automatically loads project-specific environment settings when navigating into a directory (via a `.envrc` file), and unloads them when leaving. This makes environment activation implicit and reduces the risk of running analyses in the wrong software context.

Listing 9 Building the Nix environment

```
# in the terminal, from the project directory:
nix-build
```

The expected output should look similar to (Listing 10):

Listing 10 Expected output from `nix-build`

```
unpacking 'https://github.com/rstats-on-nix/nixpkgs/archive/2025-08-25.tar.gz'
  into the Git cache...
warning: ignoring untrusted substituter...
warning: ignoring the client-specified setting...
/nix/store/qa7fq20m2f94szsnqzciwv8h4n81w43v-nix-shell
```

This command builds the environment according to the specification. The first execution will download and install all required packages, which may take a few minutes depending on network speed and system resources. Subsequent builds use cached packages and complete in seconds. Upon successful completion, a path to the constructed environment in the Nix store is printed (here, `/nix/store/qa7fq20m2f94szsnqzciwv8h4n81w43v-nix-shell`), and a symbolic link named `result` appears in the project directory pointing to this location. In our illustration, this build already includes the R packages declared for the simulation (Listing 3).

Note that the warnings indicate that you are not configured as a trusted user, so Nix cannot use the rstats-on-nix binary cache and will instead compile packages from source, which is slower. To enable binary caching, see the {rix} documentation: https://docs.ropensci.org/rix/articles/binary-cache.html.

To activate the environment, run (Listing 11):

> ⚠ Warning
>
> Do we want to nix-shell —pure?

The expected output (if you have configured yourself as a trusted user, otherwise the same

Listing 11 Activating the Nix environment

```
# in the terminal, from the project directory:
nix-shell
```

warnings will appear) should look similar to (Listing 12):

Listing 12 Expected output from `nix-shell`

```
unpacking 'https://flakehub.com/f/DeterminateSystems/nixpkgs-weekly/...'
  into the Git cache...
[nix-shell:~/Why-risk-it-when-you-can-rix-it]$
```

This command drops the user into a shell where all specified packages and tools are available. The shell prompt changes to indicate that a Nix environment is active (here, `[nix-shell:~/Why-risk-it-when-you-can-rix-it]$`). To verify that R is being provided by Nix rather than a system installation, run `which R`. This should return a path within `/nix/store/`. Moreover, from within the Nix shell, users can launch their IDE by typing its name (e.g., `rstudio` or `positron`), which opens the IDE with the Nix environment active (Listing 13):

Listing 13 Activating RStudio

```
[nix-shell:~/Why-risk-it-when-you-can-rix-it]$ rstudio
```

Note that launching RStudio this way does not automatically open the specific project you are working in. We therefore recommend creating an RStudio project file (`.Rproj`) and opening that file when using Nix, so that RStudio is correctly associated with the intended project and environment.

Step 5: Developing and Making the Simulation Reproducible

With RStudio open from the Nix environment, we write the five simulation `.R` scripts inside the `Simulation` subfolder. Note that one does not need to use RStudio, but we illustrate it as most readers probably make use of an IDE. Because we declared the simulation packages up

front (Listing 3), they are already available in the shell, and the scripts run against exactly those versions. Researchers are rarely aware of every package a project will need in advance, but, as highlighted before, packages can be added "on-the-go": as the code takes shape and calls for another package, we add it to `r_pkgs` in `gen-env.R` and rerun `rix()`. It is crucial that, after editing `gen-env.R`, we rebuild with `nix-build` and re-enter with `nix-shell` so the new packages become available.

Running the Simulation and Results

> ⚠ Warning
>
> hmm. We dont say download the R scripts. Maybe in the beginning say create a folder with two subfolders and download the R scripts for simulation and put them in there? I see no where in the ms where these scripts can be run and created which means it is not a self contained tutorial.

With the environment built, we run the simulation from inside the Nix shell so that it uses exactly the R version and packages declared in `default.nix`. This is how the results and plots reported in this manuscript were produced. For example, `03_run_simulation.R` begins by loading the required packages and sourcing the other scripts it needs:

Listing 14 Code for running the simulation

```
library(marginaleffects)
...
# Source helper functions
source("Simulation/01_data_generation.R")
...
```

The simulation needs nothing beyond the environment built above (Listing 3). From within the Nix shell, the simulation runs (Listing 15) as follows:

In the same way, we run `04_performance_metrics.R` (Listing 16), which loads the simulation results (in `sim_results.rds`) and calculates the performance metrics:

Listing 15 Running the complete simulation workflow

```
[nix-shell:~/Why-risk-it-when-you-can-rix-it]$
  Rscript Simulation/03_run_simulation.R
```

Listing 16 Running the performance metrics calculation

```
[nix-shell:~/Why-risk-it-when-you-can-rix-it]$
  Rscript Simulation/04_performance_metrics.R
```

Similarly, `05_plots.R` (Listing 17) uses those saved metrics (in `performance_summary.rds`) to create the plots:

Listing 17 Generating the visualization plots

```
[nix-shell:~/Why-risk-it-when-you-can-rix-it]$
  Rscript Simulation/05_plots.R
```

Because these results and plots come entirely from running the scripts inside the Nix environment, they are reproducible. This approach does, however, rely on running the scripts in sequence by hand. It therefore guarantees a reproducible environment but does not formalize the analytical pipeline itself, leaving the dependencies between scripts implicit in the code rather than explicitly declared.

Step 6: Developing and Making the Complete Manuscript Reproducible

To also build the manuscript, we extend the simulation environment (Listing 3) with the literate programming tools it lacks: Quarto, {knitr}, and a set of LaTeX packages. We add these arguments to `gen-env.R` and rebuild. Listing 18 shows only what is added:

Including system-level software. Nix handles non-R software in two ways. The system-level libraries your R packages depend on, such as BLAS/LAPACK, are resolved automatically and need not be listed. The {rvinecopulib} package, for example, links against the C++ libraries Boost and Eigen, which we never list. Anything else the project needs is requested explicitly through `system_pkgs`: most often command-line tools such as Git, Pandoc, and

Listing 18 Manuscript tools added to the simulation environment

```
rix(
  # ...all simulation arguments from @lst-rix-env-sim, plus:
  r_pkgs = c(..., "quarto", "knitr"), # add to the existing vector
  system_pkgs = c("quarto"), # command-line Quarto
  tex_pkgs = c(
    "amsmath",
    "ninecolors",
    "apa7",
    "scalerel",
    "threeparttable",
    "threeparttablex",
    "endfloat",
    "environ",
    "multirow",
    "tcolorbox",
    "pdfcol",
    "tikzfill",
    "fontawesome5",
    "framed",
    "newtx",
    "fontaxes",
    "xstring",
    "wrapfig",
    "tabularray",
    "siunitx",
    "fvextra",
    "geometry",
    "setspace",
    "fancyvrb",
    "anyfontsize"
  )
)
```

Quarto, though in principle any package available through Nix, including additional system libraries. The `apaquarto` extension we use for APA formatting is a separate case: it is a Quarto extension stored in the project's `_extensions/` directory rather than a Nix package (Rodrigues & Baumann, 2025) (see the {rix} documentation for more: https://docs.ropensci.org/rix/articles/installing-system-tools.html).

Specifying LaTeX packages. The `tex_pkgs` parameter lists the LaTeX packages needed for PDF compilation. When any are listed, Nix includes a minimal TeXLive distribution (`scheme-small`) and adds the requested packages on top. Finding the exact set often takes some trial and error, since Quarto's error messages during rendering indicate which packages are missing. Appendix B gives also the reasons for adding each package in `tex_pkgs`.

After rebuilding with `nix-build` and re-entering with `nix-shell`, Quarto is available. We write the manuscript (`article.qmd`) in a `Manuscript` subfolder, and render it from the terminal (Listing 19):

Listing 19 Rendering the manuscript with Quarto

```
[nix-shell:~/Why-risk-it-when-you-can-rix-it]$
  quarto render Manuscript/article.qmd
```

This command executes all code chunks in the manuscript (i.e., loads the R packages, runs the data generation and estimation), calculates and incorporates results (e.g., Table 2) and figures (e.g., Figure 1), and generates a formatted PDF following APA style guidelines via the `apaquarto` extension (Schneider, 2024). This extension is saved in the project directory already. To add it to a project of your own, run the following inside the Nix shell, where Quarto is provided by the environment and therefore needs no separate installation (Listing 20):

Listing 20 Adding the apaquarto extension

```
[nix-shell:~/Why-risk-it-when-you-can-rix-it]$
  quarto add wjschne/apaquarto
```

The final document (which can be saved as a .docx, .pdf, or .html) is saved directly in the project subfolder[10]. Because Quarto is installed as a command-line tool in our Nix specification, the rendering occurs entirely within a fully reproducible environment, ensuring consistent output

[10] Although we use `apaquarto` in this example, many alternative manuscript templates are available, and Nix is agnostic to the specific template employed, provided the necessary extensions are installed.

across machines regardless of local software configurations. If desired, the manuscript can also be reproduced interactively by opening the project folder in the user's preferred IDE.

As highlighted before, we included the simulation code as code chunks for this manuscript. This was done simply for illustration purposes. In a real-world simulation study, users could for example run the simulation as in the previous sections and then load the needed results (produced within the Nix shell) in the manuscript. However, the same remark holds with respect to the formalized workflow.

At this point, it is also worth noting that Nix shells do not fully isolate you from your existing system by default. For R users, this has a practical implication: packages installed in your regular R library (outside of Nix) could potentially be loaded when running R from within the Nix environment. The {rix} package addresses this automatically. When you call `rix()`, it also executes `rix_init()`, which creates a project-specific `.Rprofile`. This file configures R to ignore external package libraries and also disables `install.packages()` within the environment. The rationale is straightforward: any new packages should be added to `default.nix` and the environment rebuilt, preserving full reproducibility (Rodrigues & Baumann, 2025). However, for stricter isolation[11] that also prevents access to other system programs not specified in `default.nix`, use the `--pure` flag (Listing 21):

Listing 21 Activating the Nix environment with strict isolation

```
nix-shell --pure
```

A related caveat specific to macOS is discussed in the Limitations section below; see also the {rix} documentation (https://docs.ropensci.org/rix/articles/setting-up-macos.html).

[11] For example, when preparing this manuscript without the `--pure` flag, `quarto render` worked successfully. However, when using the `--pure` flag, the build failed. Running `quarto check` from within the Nix shell (i.e., `nix-shell --run "quarto check"`) revealed that Quarto was still accessing the system's LaTeX installation (`/Library/TeX/texbin`) rather than being restricted to only what was specified in `default.nix`.

Reproducing the Simulation Study and Manuscript

Having built the project as its authors, we now switch to the perspective of a reader who wants to reproduce it. Because the environment is bundled with the project, reproduction requires almost nothing beyond the project folder itself. A reader can download the entire project from its repository (https://doi.org/10.5281/zenodo.21098114). The folder already contains `default.nix`, thus they do not need the authoring steps (Steps 1 to 3). With Nix installed, they need only build and enter the environment as in Step 4 (`nix-build`, then `nix-shell`), which guarantees that all dependencies (the R version, packages, and system tools) match exactly those specified in `default.nix`.

From within that environment, reproduction proceeds on two levels. The simulation can be reproduced by running its scripts in order, regenerating the results and figures. The complete manuscript (its text together with the embedded results, tables, and figures) can be reproduced in the same environment as in Listing 19, which re-executes every code chunk and regenerates the formatted PDF end to end.

Additional Considerations for Simulation Studies

In this section, we turn to considerations that matter especially for simulation studies but fall outside this tutorial's scope, since they depend on the details of each study. The first concerns random draws and reproducibility. The second concerns the analytical pipeline, environment for multiple programming languages environment, and integration with tools researchers may already use. We do not cover these topics in depth. Our goal is to clarify how they relate to {rix} and to point readers toward resources for more advanced use cases.

Reproducible Randomness and Parallel Execution for Data Generation

A simulation regenerates its data from a pseudo-random number generator, so its reproducibility depends on controlling that generator. The standard advice is to set the seed once and store it, so the same code reproduces the same sequence of draws (Morris et al., 2019). Simulation studies are typically organized as a set of conditions, each run for many repetitions, so the unit one wants to reproduce is a single repetition within a given condition. Storing the

generator's state at the start of each condition-repetition cell, rather than only the initial seed, additionally allows any single cell to be reproduced in isolation, which is convenient for debugging (Morris et al., 2019). In R, the state lives in the variable `.Random.seed`, and such a procedure would look as follows (Listing 22):

Listing 22 Storing and restoring the generator state to reproduce one condition-repetition cell

```
# fix the starting position once
set.seed(123)

# 4 conditions, 1000 repetitions per condition
conditions <- expand.grid(n = c(50, 100), gamma2 = c(0, 0.3))
nsim <- 1000

# one slot per condition-repetition cell
states <- matrix(list(), nrow(conditions), nsim)
results <- matrix(NA_real_, nrow(conditions), nsim)

# nested loops shown for illustration
for (cond in seq_len(nrow(conditions))) {
  for (r in seq_len(nsim)) {
    # state at the start of cell (cond, r)
    states[[cond, r]] <- .Random.seed
    # data generation advances the state
    x <- rnorm(conditions$n[cond])
    results[cond, r] <- mean(x)
  }
}

# later, reproduce repetition 3 of condition 2 on its own
.Random.seed <- states[[2, 3]]
identical(mean(rnorm(conditions$n[2])), results[2, 3]) # TRUE
```

With a sufficient number of repetitions the particular seed does not affect the conclusions. However, parallel processing complicates this (Morris et al., 2019; Siepe et al., 2024). In this case, a single seed may no longer be sufficient, because each worker would advance its own copy of the generator and could draw overlapping sequences. In practice, many researchers assign a distinct seed to each condition, for example `set.seed(123 + i)` for condition `i`, and let the

repetitions within that condition draw sequentially from it. This keeps every condition reproducible and might not be a problem for most studies (given the properties of the default algorithm, i.e., Mersenne-Twister), but it does not by itself *guarantee* that the random sequences of different conditions never overlap. The recommended solution is to assign separate, non-overlapping random number streams (Morris et al., 2019; Siepe et al., 2024). R offers several packages for this (Gaujoux, 2025; Leydold, 2022; R Core Team, 2023), and we illustrate the approach with {doRNG}, which implements streams for {foreach} through the `%dorng%` operator (Listing 14). From a single master seed set before the loop, {doRNG} derives one independent stream per iteration, bound to the iteration index rather than to a worker. The number of cores (`ncore` in our script) therefore changes only how iterations are scheduled, never which stream each receives, so a collaborator who reruns the study on four cores reproduces data generated on twenty cores exactly. Because {doRNG} also records each stream in the result's `rng` attribute, these states can be stored and later restored to reproduce any single repetition in isolation, which the per-condition seed alone cannot do (Listing 23). Researchers should store these states and, preferably, make them available alongside the reported estimates.

Analytical Pipelines: {targets} and {rixpress}

Complex simulation studies often benefit from an analytical pipeline that tracks dependencies between computational steps, caches intermediate results, and enables selective re-execution when inputs change. Two complementary approaches exist within the Nix ecosystem.

Using {targets} Within Nix. The {targets} package (Landau, 2021) provides analytical pipeline tooling for R-based projects. To integrate {targets} with Nix, include `targets` in the `r_pkgs` parameter of `rix()` and execute the pipeline within `nix-shell` using `Rscript -e 'targets::tar_make()'`. A shell hook can also be added via the `shell_hook` argument to run the pipeline automatically when entering the Nix shell. This approach is ideal for projects that remain within R and do not require different environments for different pipeline steps (see {rix} documentation: https://docs.ropensci.org/rix/articles/reproducible-pipelines.html).

Listing 23 Reproducible parallel streams with {doRNG}

```
library(doRNG)

# makes the streams below reproducible
set.seed(123)

# one slot per condition
rng_states <- vector("list", nrow(designs))

# nested loop over conditions
for (k in seq_len(nrow(designs))) {
  res <- foreach(
    i = seq_len(nsim),
    .packages = c("marginaleffects", "rvinecopulib")
  ) %dorng%
    {
      run_one_rep(designs$n[k], designs$gamma2[k])
    }
  # per-repetition stream states for this condition
  rng_states[[k]] <- attr(res, "rng")
  all_results[[k]] <- do.call(rbind, res)
}

# later, reproduce repetition 10 of condition 3 on its own
RNGkind("L'Ecuyer-CMRG")
.Random.seed <- rng_states[[3]][[10]]
rep10 <- run_one_rep(designs$n[3], designs$gamma2[3])
```

Using {rixpress} for Multi-Language Pipelines. The {rixpress} package (Rodrigues, 2025), a sister package to {rix}, uses Nix itself as the build automation tool rather than operating within a Nix environment. Each pipeline step becomes a Nix derivation, built in isolation with automatic caching based on content. The key advantage emerges in multi-language workflows: different steps can execute in different Nix-defined environments (e.g., one step using a specific version of R, another using Python, another using Julia). The interface, inspired by {targets}, uses functions like `rxp_r()`, `rxp_py()`, and `rxp_jl()` to define pipeline steps (see {rixpress} documentation: https://docs.ropensci.org/rixpress/articles/intro-concepts.html). The GitHub repository for this article directs interested readers to a demonstration of {rixpress} applied to this

entire project.

Multi-Language Environment Support

While this tutorial focuses on R, researchers working across multiple programming languages can include Python or Julia in their environments. The `py_conf` parameter accepts a list specifying a Python version and required packages (Listing 24). Similarly, `jl_conf` enables Julia package installation. This capability is particularly useful, for example, for projects requiring statistical computing in R alongside machine learning pipelines in Python or numerical optimization in Julia (Rodrigues & Baumann, 2025) (see {rix} documentation for more: https://docs.ropensci.org/rix/articles/installing-r-packages.html).

Listing 24 Including Python packages

```
py_conf = list(py_version = "3.12", py_pkgs = c("polars", "pandas"))
```

Converting Existing {renv} Projects

Researchers with existing {renv} projects can migrate using the `renv2nix()` function, which reads an `renv.lock` file and generates an equivalent Nix expression. This is particularly valuable for projects where {renv} encountered system dependency issues or where stricter reproducibility guarantees are desired. Unlike {renv}, which captures R package versions but not the R interpreter or system-level libraries, Nix manages all layers of the software stack (see {rix} documentation: https://docs.ropensci.org/rix/articles/renv2nix.html).

Containerization with Docker

Nix and Docker are not necessarily mutually exclusive (Rodrigues & Baumann, 2026). Researchers already using Docker do not need to abandon it to benefit from Nix, as the two can be combined by using Nix inside Docker containers to handle environment setup (Rodrigues & Baumann, 2025). This is particularly useful for deployment to cloud platforms or high-performance computing clusters where Nix may not be installed. In that case, the environment is built with Nix but shipped inside a Docker image, so it can be run with Docker

alone on machines where Nix is not installed (see {rix} documentation: https://docs.ropensci.org/rix/articles/nix-inside-docker.html).

Discussion

Reproducibility in computational research is often treated as a matter of transparency: making data and code available. This tutorial has argued that transparency alone is insufficient without the ability to reliably reconstruct the computational environments in which analyses are executed. For simulation studies in particular, where results depend critically on software versions, system-level libraries, and random number generation, environment-level reproducibility is not optional but essential.

By introducing Nix and the {rix} package, we demonstrated a practical and accessible approach to fully specifying and rebuilding computational environments for simulation-based research. This approach greatly increases the chances that analyses and manuscripts can be reproduced across machines and over time, moving reproducibility from an aspirational goal toward a verifiable property of the research workflow.

Importantly, adopting environment reproducibility does not require abandoning existing analytic practices. Nix is agnostic to programming language, editor, workflow structure, and manuscript template, allowing researchers to retain familiar tools while strengthening the reliability of their work. In this sense, reproducible environments serve as enabling infrastructure, supporting rather than replacing other best practices such as version control, analytical pipelines, and transparent reporting.

Limitations

The approach we describe has limitations worth noting, the most relevant of which is specific to macOS. There, some parts of an environment still depend on Apple's own system software, which lies outside the Nix store and is therefore not pinned. As a consequence, an environment pinned to an older date can occasionally stop building or loading after a macOS update. Because reproducing an environment faithfully means keeping its pin unchanged, the reliable solution is not to update the pin but to build and run the original environment through

Docker or on Linux (including WSL on Windows), where it no longer depends on the host's macOS (Rodrigues & Baumann, 2026). This preserves exact reproducibility while working around the platform's constraints (for macOS-specific setup, see https://docs.ropensci.org/rix/articles/setting-up-macos.html).

Finally, a reproducible environment does not, on its own, make the workflow itself transparent. This is particularly relevant for simulation studies, which are often split across several files whose dependencies, and the order in which they should run, are not always clear (for example, which script must run first). Dedicated analytical pipeline tools can formalize these dependencies (see above), but at a minimum, clearly documenting how the files relate and in what order they run already helps others reproduce the study.

Moreover, as highlighted, there may be components on simulation studies that might be hard to reproduce by others researchers such as very computationally intensive studies that demand a huge amount of computer power. The data generation also might not yield identifical numerical results due to the interplay of multiple things. However, making availble the obtained estimates, together with the pinned enrivonment, would at least allow the reproducibility of the analyses etc.

Conclusion

If reproducibility is to function as a cornerstone of cumulative science, then the ability to reconstruct computational environments must become a routine part of methodological practice. Tools such as Nix and {rix} lower the barrier to achieving this goal, making fully reproducible simulation research feasible. We hope this tutorial helps normalize environment-level reproducibility as a standard component of rigorous computational research in psychology and beyond.

References

Adler, J. (2012). *R in a nutshell* (2nd ed.). O'Reilly Media.

Ahrens, W., Demmel, J., & Nguyen, H. D. (2020). Algorithms for efficient reproducible floating point summation. *ACM Transactions on Mathematical Software*, *46*(3). https://doi.org/10.1145/3389360

Arel-Bundock, V., Greifer, N., & Heiss, A. (2024). How to interpret statistical models using marginaleffects for R and Python. *Journal of Statistical Software*, *111*(9), 1–32. https://doi.org/10.18637/jss.v111.i09

Baker, D. H., Berg, M., Hansford, K., Quinn, B., Segala, F., & Warden-English, E. (2024). ReproduceMe: Lessons from a pilot project on computational reproducibility. *Meta-Psychology*, *8*, MP.2023.4021. https://doi.org/10.15626/MP.2023.4021

Boettiger, C. (2015). An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, *49*(1), 71–79. https://doi.org/10.1145/2723872.2723882

Boettiger, C., & Eddelbuettel, D. (2017). An introduction to Rocker: Docker containers for R. *The R Journal*, *9*(2), 527–536. https://doi.org/10.32614/RJ-2017-065

Bouma, A., Assen, M. A. L. M. van, Aert, R. C. M. van, & Voncken, L. (2026). Reporting practices, open science practices, and trustworthiness of simulation studies in psychology: A questionnaire study. *Advances in Methods and Practices in Psychological Science*.

Brodeur, A., Mikola, D., Cook, N., Fiala, L., Brailey, T., Briggs, R., Gendre, A. de, Dupraz, Y., Gabani, J., Gauriot, R., Haddad, J., Lima, G., Ankel-Peters, J., Dreber, A., Campbell, D., Kattan, L., Marino Fages, D., Mierisch, F., Sun, P., … Strobel, S. (2026). Reproducibility and robustness of economics and political science research. *Nature*, *652*(8108), 151–156. https://doi.org/10.1038/s41586-026-10251-x

Corporation, M., & Weston, S. (2022). *doParallel: Foreach parallel adaptor for the 'parallel' package*. https://CRAN.R-project.org/package=doParallel

Dolstra, E., De Jonge, M., & Visser, E. (2004). Nix: A safe and policy-free system for software deployment. *18th Large Installation System Administration Conference*, 79–92.

Epskamp, S. (2019). Reproducibility and replicability in a fast-paced methodological world. *Advances in Methods and Practices in Psychological Science*, *2*(2), 145–155. https://doi.org/10.1177/2515245919847421

Feldman, S. I. (1979). Make — a program for maintaining computer programs. *Software: Practice and Experience*, *9*(4), 255–265. https://doi.org/10.1002/spe.4380090402

Gaujoux, R. (2025). *doRNG: Generic reproducible parallel backend for 'foreach' loops*. https://CRAN.R-project.org/package=doRNG

Glatard, T., Lewis, L. B., Ferreira da Silva, R., Adalat, R., Beck, N., Lepage, C., Rioux, P., Rousseau, M.-É., Sherif, T., Deelman, E., Khalili-Mahani, N., & Evans, A. C. (2015). Reproducibility of neuroimaging analyses across operating systems. *Frontiers in Neuroinformatics*, *9*, 12. https://doi.org/10.3389/fninf.2015.00012

Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, *23*(1), 5–48. https://doi.org/10.1145/103162.103163

Hardwicke, T. E., Wallach, J. D., Kidwell, M. C., Bendixen, T., Crüwell, S., & Ioannidis, J. P. A. (2020). An empirical assessment of transparency and reproducibility-related research practices in the social sciences (2014–2017). *Royal Society Open Science*, *7*(2), 190806. https://doi.org/10.1098/rsos.190806

Hodges, C. B., Stone, B. M., Johnson, P. K., Carter, J. H., III, Sawyers, C. K., Roby, P. R., & Lindsey, H. M. (2023). Researcher degrees of freedom in statistical software contribute to unreliable results: A comparison of nonparametric analyses conducted in SPSS, SAS, Stata, and R. *Behavior Research Methods*, *55*(6), 2813–2837. https://doi.org/10.3758/s13428-022-01932-2

Joshi, M., & Pustejovsky, J. (2025). *Simhelpers: Helper functions for simulation studies*. https://CRAN.R-project.org/package=simhelpers

Kidwell, M. C., Lazarević, L. B., Baranski, E., Hardwicke, T. E., Piechowski, S., Falkenberg, L.-S., Kennett, C., Slowik, A., Sonnleitner, C., Hess-Holden, C., Errington, T. M., Fiedler, S., & Nosek, B. A. (2016). Badges to acknowledge open practices: A simple, low-cost, effective

method for increasing transparency. *PLOS Biology*, *14*(5), e1002456. https://doi.org/10.1371/journal.pbio.1002456

Lamb, C., & Zacchiroli, S. (2022). Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, *39*(2), 62–70. https://doi.org/10.1109/ms.2021.3073045

Landau, W. M. (2021). *Targets: Dynamic function-oriented make-like declarative workflows*.

Levenstein, M. C., & Lyle, J. A. (2018). Data: Sharing is caring. *Advances in Methods and Practices in Psychological Science*, *1*(1), 95–103.

Leydold, J. (2022). *Rstream: Streams of random numbers*. https://CRAN.R-project.org/package=rstream

Luijken, K., Lohmann, A., Alter, U., Claramunt Gonzalez, J., Clouth, F. J., Fossum, J. L., Hesen, L., Huizing, A. H. J., Ketelaar, J., Montoya, A. K., Nab, L., Nijman, R. C. C., Penning de Vries, B. B. L., Tibbe, T. D., Wang, Y. A., & Groenwold, R. H. H. (2024). Replicability of simulation studies for the investigation of statistical methods: The RepliSims project. *Royal Society Open Science*, *11*(1), 231003. https://doi.org/10.1098/rsos.231003

Malka, J., Zacchiroli, S., & Zimmermann, T. (2024). Reproducibility of build environments through space and time. *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, 97–101. https://doi.org/10.1145/3639476.3639767

Malka, J., Zacchiroli, S., & Zimmermann, T. (2026). Docker does not guarantee reproducibility. *arXiv Preprint arXiv:2601.12811*. https://arxiv.org/abs/2601.12811

Montoya, A. K., & Anderson, S. F. (2026). Repeatability in monte carlo simulation studies. *Advances in Methods and Practices in Psychological Science*.

Morris, T. P., White, I. R., & Crowther, M. J. (2019). Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, *38*(11), 2074–2102. https://doi.org/10.1002/sim.8086

Nagler, T., & Vatter, T. (2025). *Rvinecopulib: High performance algorithms for vine copula modeling*. https://CRAN.R-project.org/package=rvinecopulib

Nosek, B. A., Alter, G., Banks, G. C., Borsboom, D., Bowman, S. D., Breckler, S. J., Buck, S., Chambers, C. D., Chin, G., Christensen, G., Contestabile, M., Dafoe, A., Eich, E., Freese, J., Glennerster, R., Goroff, D., Green, D. P., Hesse, B., Humphreys, M., … Yarkoni, T. (2015). Promoting an open research culture. *Science*, *348*(6242), 1422–1425. https://doi.org/10.1126/science.aab2374

Nosek, B. A., Hardwicke, T. E., Moshontz, H., Allard, A., Corker, K. S., Dreber, A., Fidler, F., Hilgard, J., Kline Struhl, M., Nuijten, M. B., Rohrer, J. M., Romero, F., Scheel, A. M., Scherer, L. D., Schönbrodt, F. D., & Vazire, S. (2022). Replicability, robustness, and reproducibility in psychological science. *Annual Review of Psychology*, *73*, 719–748. https://doi.org/10.1146/annurev-psych-020821-114157

Ottoboni, K., & Stark, P. B. (2018). *Random problems with r*. https://arxiv.org/abs/1809.06520

Pawel, S., Bartoš, F., Siepe, B. S., & Lohmann, A. (2025). Handling missingness, failures, and non-convergence in simulation studies: A review of current practices and recommendations. *The American Statistician*, 1–18. https://doi.org/10.1080/00031305.2025.2540002

Pawel, S., Kook, L., & Reeve, K. (2024). Pitfalls and potentials in simulation studies: Questionable research practices in comparative simulation studies allow for spurious claims of superiority of any method. *Biometrical Journal*, *66*(1), 2200091. https://doi.org/10.1002/bimj.202200091

Peikert, A., & Brandmaier, A. M. (2021). A reproducible data analysis workflow. *Quantitative and Computational Methods in Behavioral Sciences*, *1*, e3763. https://doi.org/10.5964/qcmb.3763

Peikert, A., van Lissa, C. J., & Brandmaier, A. M. (2021). Reproducible research in R: A tutorial on how to do the same thing more than once. *Psych*, *3*(4), 836–867. https://doi.org/10.3390/psych3040053

Peng, R. D. (2011). Reproducible research in computational science. *Science*, *334*(6060), 1226–1227.

R Core Team. (2023). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. https://www.R-project.org/

R Core Team. (2025). *R installation and administration*. R Foundation for Statistical Computing. https://cran.r-project.org/doc/manuals/r-release/R-admin.html

Rodrigues, B. (2023). *Building reproducible analytical pipelines with R*. https://raps-with-r.dev

Rodrigues, B. (2025). *Rixpress: Build reproducible analytical pipelines with 'nix'*. https://CRAN.R-project.org/package=rixpress

Rodrigues, B., & Baumann, P. (2025). *Rix: Reproducible data science environments with 'nix'*. https://CRAN.R-project.org/package=rix

Rodrigues, B., & Baumann, P. (2026). *Nix for polyglot, reproducible data science workflows* (Version v0.0.1). Zenodo. https://doi.org/10.5281/zenodo.18138618

Schneider, W. J. (2024). *Apaquarto* [Computer software]. https://github.com/wjschne/apaquarto

Siepe, B. S., Bartoš, F., Morris, T. P., Boulesteix, A.-L., Heck, D. W., & Pawel, S. (2024). Simulation studies for methodological research in psychology: A standardized template for planning, preregistration, and reporting. *Psychological Methods*. https://doi.org/10.1037/met0000695

Simonsohn, U. (2020). *Groundhog: Version-control for CRAN, github, and gitlab packages*.

Stan Development Team. (2025). *Reproducibility*. https://mc-stan.org/docs/reference-manual/reproducibility.html

Ushey, K. (2024). *Renv: Project environments*.

van Lissa, C. J., Brandmaier, A. M., Brinkman, L., Lamprecht, A.-L., Peikert, A., Struiksma, M. E., & Vreede, B. M. I. (2021). WORCS: A workflow for open reproducible code in science. *Data Science*, *4*(1), 29–49. https://doi.org/10.3233/DS-210031

Vazire, S. (2018). Implications of the credibility revolution for productivity, creativity, and progress. *Perspectives on Psychological Science*, *13*(4), 411–417. https://doi.org/10.1177/1745691617751884

White, I. R., Pham, T. M., Quartagno, M., & Morris, T. P. (2024). How to check a simulation study. *International Journal of Epidemiology*, *53*(1), dyad134. https://doi.org/10.1093/ije/dyad134

Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer-Verlag New York. https://ggplot2.tidyverse.org

Wickham, H., François, R., Henry, L., Müller, K., & Vaughan, D. (2023). *Dplyr: A grammar of data manipulation*. https://CRAN.R-project.org/package=dplyr

Wiebels, K., & Moreau, D. (2021). Leveraging containers for reproducible psychological research. *Advances in Methods and Practices in Psychological Science*, *4*(2). https://doi.org/10.1177/25152459211017853

Wilke, C. O. (2025). *Cowplot: Streamlined plot theme and plot annotations for 'ggplot2'*. https://wilkelab.org/cowplot/

Ziemann, M., Poulain, P., & Bora, A. (2023). The five pillars of computational reproducibility: Bioinformatics and beyond. *Briefings in Bioinformatics*, *24*(6), bbad375. https://doi.org/10.1093/bib/bbad375

Table 1
Glossary of computing terms used in the main text.

Term	Definition
System-level libraries	Shared operating-system software that R packages depend on (e.g., BLAS/LAPACK, C++ backends). These are not installed by `install.packages()`.
Operating system	The base software (e.g., Windows, macOS, Linux) that manages a computer's hardware and on which every other program runs. R, its packages, and system-level libraries all sit on top of it.
R interpreter	The `R` program that reads and runs your R code (Adler, 2012). Two machines can report the same R version yet have interpreters compiled differently or linked against different system-level libraries (see R Core Team, 2025, Section A.3).
Analytical pipeline	Automating the order in which analysis steps run and the dependencies among them, re-running only those affected by a change.
Software stack	The complete set of software layers an analysis depends on: the operating system, system-level libraries, the R interpreter, and the R packages built on top of them.
Containerization	An isolated, lightweight, shareable bundle of software and its dependencies (e.g., Docker) (Wiebels & Moreau, 2021).
Mutable upstream repositories	The external sources a build pulls from (e.g., base images or package repositories) can change, or even disappear, over time without the build instructions changing, so the same instructions may later produce a different result or fail to build (Malka et al., 2026).
Reproducible workflow	The overall research process that makes a study reproducible end to end, including version control, the analytical pipeline, and a frozen computational environment.
Literate programming	Writing an analysis as a single document that combines narrative text and executable code (e.g., R Markdown or Quarto), so the report and the code that produces it stay together.
Functional programming language	A language built around pure functions, immutability, and explicit inputs, so the same inputs always produce the same output.
Binary cache	A server that stores prebuilt packages, so Nix can download a finished copy instead of compiling it from source.
cryptographic hash	define

Table 2

Performance metrics for ACE estimator across simulation conditions. Values in parentheses are Monte Carlo standard errors (MCSE).

Sample Size	Confounding	Relative Bias	Relative RMSE	Coverage	CI Width
50	none	0.961 (0.039)	0.388 (0.037)	0.920 (0.027)	0.226 (0.004)
100	none	1.017 (0.027)	0.265 (0.022)	0.950 (0.022)	0.159 (0.002)
2000	none	0.997 (0.006)	0.059 (0.005)	0.950 (0.022)	0.036 (0.000)
50	mild	1.150 (0.043)	0.457 (0.040)	0.860 (0.035)	0.226 (0.005)
100	mild	1.080 (0.028)	0.285 (0.023)	0.880 (0.032)	0.159 (0.002)
2000	mild	1.127 (0.006)	0.142 (0.006)	0.470 (0.050)	0.035 (0.000)
50	severe	1.219 (0.045)	0.500 (0.043)	0.870 (0.034)	0.226 (0.004)
100	severe	1.230 (0.032)	0.393 (0.026)	0.820 (0.038)	0.156 (0.002)
2000	severe	1.238 (0.007)	0.247 (0.007)	0.050 (0.022)	0.035 (0.000)

Figure 1

The computational environment as a set of nested layers, based on Peikert & Brandmaier (2021).

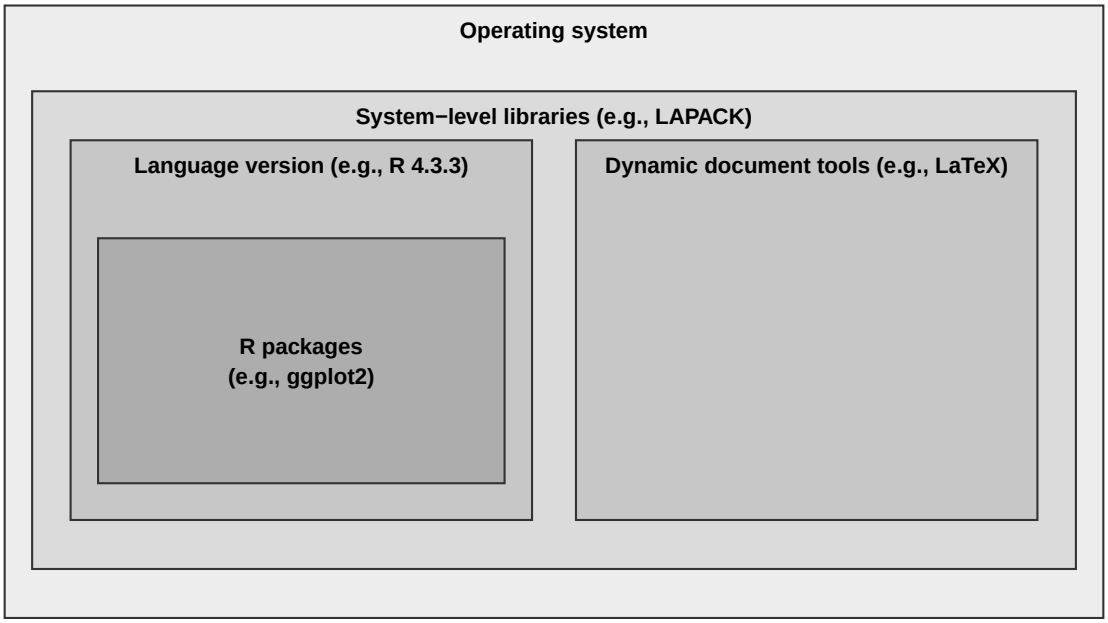
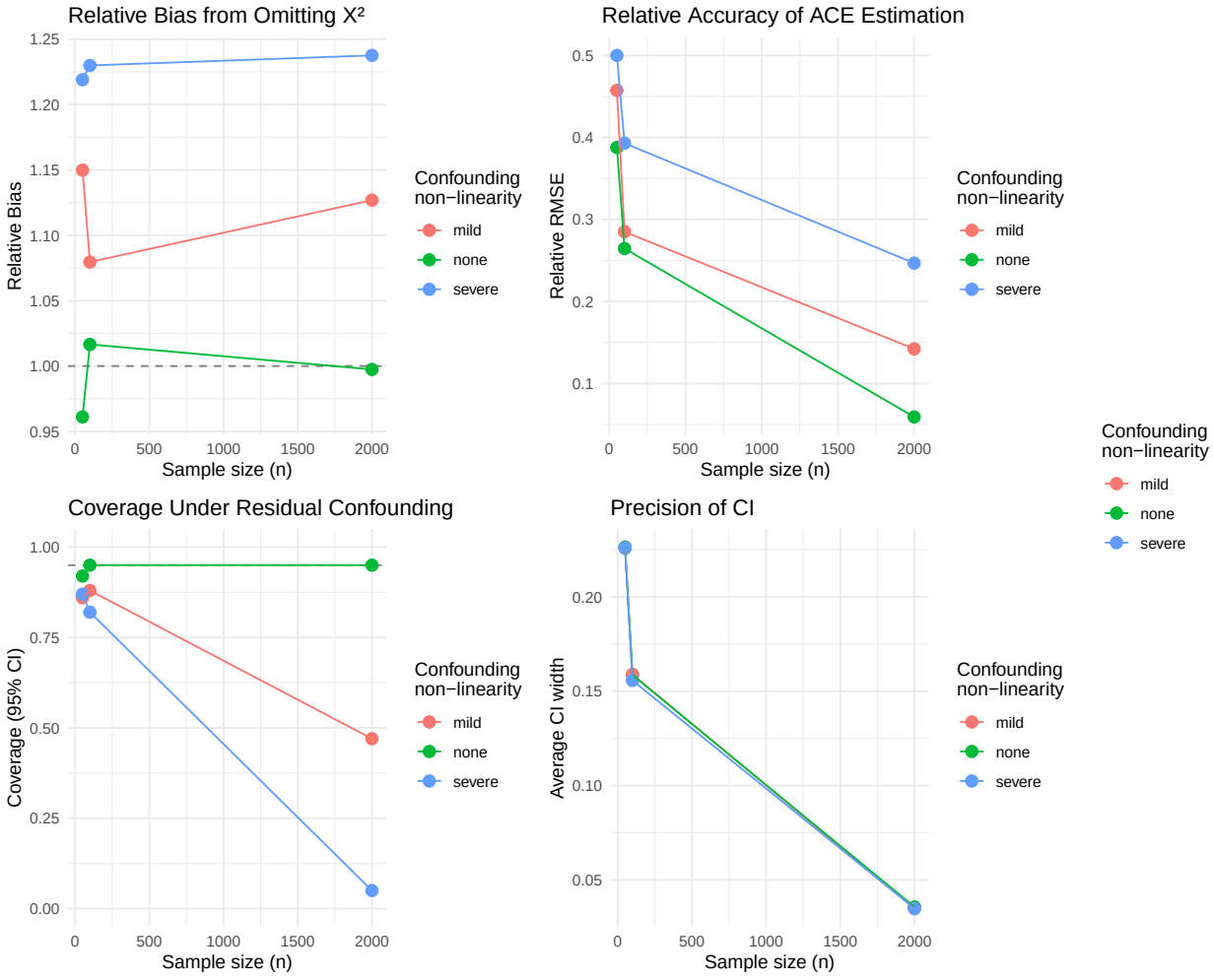


Figure 2

Performance of ACE estimator across sample sizes and confounding severity. Panel A shows relative bias, Panel B shows relative RMSE, Panel C shows coverage probability of 95% confidence intervals (dashed line at nominal 0.95 level), and Panel D shows average confidence interval width. Results demonstrate that model misspecification induces systematic bias that persists across sample sizes, while increasing sample size improves precision but not accuracy under misspecification.



Appendix A

Simulation Study Design

Here we present a rather short description following recommendations from previous research, but ideally even more may be reported (Morris et al., 2019; Pawel et al., 2025; Siepe et al., 2024; White et al., 2024). This mimics a methods or similar section in articles.

Factorial Design. The simulation employs a full factorial design with two factors: sample size ($n \in \{50, 100, 2000\}$) and degree of confounding non-linearity ($\gamma_2 \in \{0, 0.3, 0.8\}$, labeled as none, mild, and severe). The parameter γ_2 controls the strength of the quadratic confounder effect on the outcome (see Data Generation). This yields nine conditions, each replicated $K = 100$ times.

Data Generation. For each replication, data are generated following a causal structure where a confounder X_2 affects both treatment assignment and the outcome. The confounder and treatment error term are generated using the {rvinecopulib} package: pairs (U_1, U_2) are drawn from an independence copula via `rbicop()`, then transformed to standard normals via $X_2 = \Phi^{-1}(U_1)$ and $\epsilon = \Phi^{-1}(U_2)$. The independence copula is simply $C(u, v) = uv$, meaning the resulting uniforms are independent—mathematically equivalent to calling `rnorm()` directly. We use {rvinecopulib} intentionally because it depends on C++ libraries.

Treatment assignment follows $X_1 = \alpha_0 + \alpha_1 X_2 + \alpha_2 X_2^2 + \epsilon$ where $\alpha_0 = 0$, $\alpha_1 = 0.5$, and $\alpha_2 = 0.2$. This creates confounding because X_2 influences treatment assignment through both linear and quadratic terms. The binary outcome is generated from the true logistic regression model:

$$\text{logit}(P(Y = 1 \mid X_1, X_2)) = \beta_0 + \beta_1 X_1 + \gamma_1 X_2 + \gamma_2 X_2^2$$

with parameters $\beta_0 = -0.5$, $\beta_1 = 0.7$ (the causal effect of interest), $\gamma_1 = -0.4$, and γ_2 varying by condition. The analyst misspecifies the outcome model by omitting the quadratic confounder term, fitting instead:

$$\text{logit}(P(Y = 1 \mid X_1, X_2)) = \beta_0 + \beta_1 X_1 + \gamma_1 X_2$$

This misspecification creates residual confounding because the omitted term $\gamma_2 X_2^2$ is

correlated with X_1 (since X_1 depends on both X_2 and X_2^2), violating the conditional exchangeability assumption given linear adjustment alone.

Estimand. The target estimand is the average causal effect (ACE) of X_1 on Y, properly adjusted for confounding:

$$\text{ACE}(X_1) = \mathbb{E}\left[\frac{\partial P(Y = 1 \mid X_1, X_2)}{\partial X_1}\right] = \mathbb{E}\left[\beta_1 \cdot \frac{\exp(\eta)}{(1 + \exp(\eta))^2}\right]$$

where $\eta = \beta_0 + \beta_1 X_1 + \gamma_1 X_2 + \gamma_2 X_2^2$ is the correctly specified linear predictor, and the expectation is taken over the joint distribution of (X_1, X_2). For each γ_2 condition, the "true" ACE (denoted θ) is approximated once using a very large sample ($N = 200,000$) with the correctly specified model including X_2^2.

Estimator. The causal effect is estimated from the misspecified model (omitting X_2^2) as:

$$\widehat{\text{ACE}}(X_1) = \frac{1}{n}\sum_{i=1}^{n} \tilde{\beta}_1 \cdot \frac{\exp(\tilde{\eta}_i)}{(1 + \exp(\tilde{\eta}_i))^2}$$

where $\tilde{\eta}_i = \tilde{\beta}_0 + \tilde{\beta}_1 X_{i1} + \tilde{\gamma}_1 X_{i2}$ and $\tilde{\boldsymbol{\beta}} = (\tilde{\beta}_0, \tilde{\beta}_1, \tilde{\gamma}_1)$ are maximum likelihood estimates from the misspecified logistic regression.

Performance Criteria. Table B1 presents the performance criteria used to evaluate the ACE estimator across simulation conditions.

Computational Details. The simulation was conducted on a MacBook Pro (Apple M4 Pro Chip), running macOS Sequoia 15.7.3. All analyses were performed in R (version 4.5.2). Parallel processing was implemented through the {doParallel} package [version 1.0.17; Corporation and Weston (2022)], with {doRNG} [version 1.8.6.2; Gaujoux (2025)] to ensure independent and reproducible random number streams. For data generation we used the {rvinecopulib} package [version 0.7.3.1.0; Nagler and Vatter (2025)]. The estimator was implemented using the {marginaleffects} package [version 0.31.0; Arel-Bundock et al. (2024)]. Data wrangling was performed with {dplyr} [version 1.1.4; Wickham et al. (2023)]. Method performance was assessed through multiple metrics following the formulas from the {simhelpers}

package [version 0.3.1; Joshi and Pustejovsky (2025)]. Figures were produced with {ggplot2} [version 4.0.1; Wickham (2016)] and {cowplot} [version 1.2.0; Wilke (2025)].

Appendix B

Packages Clarification

R Packages

Reproducibility Infrastructure

rix: Generates the Nix expression (`default.nix`) for reproducible environments.

Simulation Study

These packages are used in the `Simulation/` folder:

rvinecopulib (used in `01_data_generation.R`): Generates correlated data via copulas using the `rbicop()` function.

marginaleffects (used in `02_models.R`): Computes average causal effects via the `avg_slopes()` function.

doParallel (used in `03_run_simulation.R`): Enables parallel `foreach` loops across CPU cores.

doRNG (used in `03_run_simulation.R`): Makes parallel RNG reproducible.

simhelpers (used in `04_performance_metrics.R`): Calculates bias, RMSE, and coverage metrics.

ggplot2 (used in `05_plots.R`): Creates simulation result visualizations.

cowplot (used in `05_plots.R`): Combines plots with `plot_grid()` and extracts legends.

dplyr (used in `article.qmd`): Data wrangling for results reported in Table 2.

Dynamic Document Generation

quarto: R interface to invoke Quarto rendering.

knitr: Executes R code chunks in `.qmd` files.

svglite: SVG graphics device; apaquarto sets `dev: svglite` for HTML output.

LaTeX Packages

Required by apaquarto Extension

These are loaded in apaquarto's `header.tex` or `apatemplate.tex`:

amsmath: Math environments (`align`, `equation`, etc.).

threeparttablex: Tables with notes below (APA table format).

tcolorbox: Callout boxes (note, warning, tip blocks).

fontawesome5: Icons in callouts.

multirow: Table cells spanning multiple rows.

newtx: Times-like fonts (default when no custom `mainfont`).

Dependencies of apaquarto Packages

environ: Dependency of tcolorbox.

pdfcol: Dependency of tcolorbox.

tikzfill: Dependency of tcolorbox.

fontaxes: Dependency of newtx.

xstring: Dependency of newtx.

scalerel: Dependency of apa7 class.

Required by apa7 Document Class

apa7: The document class itself (`\documentclass{apa7}`).

endfloat: Moves floats to end of document in manuscript mode.

threeparttable: Dependency for threeparttablex.

geometry: Page margins.

Required by Quarto PDF Rendering

framed: Shaded/framed regions for callouts.

fvextra: Enhanced verbatim for syntax-highlighted code blocks.

fancyvrb: Verbatim environments for code display.

setspace: Line spacing; also used in `article.qmd` for single-spaced code blocks.

anyfontsize: Arbitrary font sizes in code blocks.

Additional Packages

ninecolors: Extended color palettes.

wrapfig: Text wrapping around figures.

tabularray: Modern table typesetting.

siunitx: SI units and number formatting.

Table B1

Performance criteria for evaluating the ACE estimator. $\hat{\theta}_k$ denotes the ACE estimate from replication k (for $k = 1, \ldots, K$), where $K = 1000$ is the number of replications, and θ denotes the true ACE for a given condition. For coverage and width criteria, A_k and B_k denote the lower and upper endpoints of the 95% confidence interval from replication k, $W_k = B_k - A_k$ is the interval width, c_β is the estimated coverage probability, and $I(\cdot)$ is an indicator function equaling 1 if the condition is true and 0 otherwise. The Monte Carlo standard error (MCSE) quantifies the simulation uncertainty in each performance measure estimate

Criterion	Estimate	MCSE
Bias	$\frac{1}{K}\sum_{k=1}^{K}\hat{\theta}_k - \theta$	$\sqrt{\frac{1}{K(K-1)}\sum_{k=1}^{K}(\hat{\theta}_k - \bar{\hat{\theta}})^2}$
Variance	$\frac{1}{K-1}\sum_{k=1}^{K}(\hat{\theta}_k - \bar{\hat{\theta}})^2$	$\sqrt{\frac{K-1}{K}} \cdot \frac{1}{K-1}\sum_{k=1}^{K}(\hat{\theta}_k - \bar{\hat{\theta}})^2$
RMSE	$\sqrt{\frac{1}{K}\sum_{k=1}^{K}(\hat{\theta}_k - \theta)^2}$	$\sqrt{\frac{K-1}{K}\sum_{j=1}^{K}\left(\sqrt{(\hat{\theta}_j - \theta)^2} - \overline{RMSE}\right)^2}$
Relative Bias	$\frac{1}{\theta K}\sum_{k=1}^{K}\hat{\theta}_k$	$\frac{1}{\theta}\sqrt{\frac{1}{K(K-1)}\sum_{k=1}^{K}(\hat{\theta}_k - \bar{\hat{\theta}})^2}$
Relative RMSE	$\frac{1}{\theta}\sqrt{\frac{1}{K}\sum_{k=1}^{K}(\hat{\theta}_k - \theta)^2}$	$\frac{1}{\theta}\sqrt{\frac{K-1}{K}\sum_{j=1}^{K}\left(\sqrt{(\hat{\theta}_j - \theta)^2} - \overline{RMSE}\right)^2}$
Coverage	$\frac{1}{K}\sum_{k=1}^{K} I(A_k \leq \theta \leq B_k)$	$\sqrt{\frac{c_\beta(1-c_\beta)}{K}}$
Width	$\frac{1}{K}\sum_{k=1}^{K}(B_k - A_k)$	$\sqrt{\frac{1}{K(K-1)}\sum_{k=1}^{K}(W_k - \bar{W})^2}$